

Design and Implementation of an Autonomous Line Following Food Delivery Robot for Smart Restaurant Services

Peripheral Thesis Project

Submitted to
Department of Computer Science and Engineering
Port City International University

Submitted By

Sreemanta Bonik **MD. Nayeemur Rashid**
ID: CSE-031 07972 ID: CSE-031 07977

Syed MD. Sabbir
ID: CSE-031 07966

Apurbo Das **MD. Anowar Hossain**
ID: CSE-031 08012 ID: CSE-031 07985

Supervisor: K. M. Tousif bin Parves

Designation: Lecturer

Department: Department of Computer Science and Engineering

Faculty: Faculty of Science and Engineering

University: Port City International University

*A thesis submitted in partial fulfillment of the requirements for the degree of Bachelor of
Science in Computer Science and Engineering.*

14 May 2026

Acknowledgement

First and foremost, we express our sincere gratitude to Almighty Allah for granting us the strength, patience, and opportunity to complete this thesis successfully.

We would like to express our heartfelt appreciation to our respected supervisor for his continuous support, valuable guidance, and technical assistance throughout the development of this project.

We are also thankful to the Department of Computer Science and Engineering of Port City International University for providing academic support and laboratory facilities during this research work.

Finally, we express our gratitude to our friends, classmates, and family members for their constant encouragement and support.

Abstract

Automation and autonomous robotic systems are becoming increasingly important in modern industries and smart service sectors.

This thesis presents the design and implementation of an autonomous line following food delivery robot for smart restaurant services using a 5-channel infrared sensor array and Arduino Nano microcontroller.

The proposed robot follows predefined navigation paths autonomously while maintaining stable movement through real-time sensor feedback and PD-based error correction.

The developed system consists of an Arduino Nano, L298N motor driver, N20 geared motors, lithium-ion battery system, and lightweight chassis.

Experimental results demonstrate that the robot can efficiently navigate both straight and curved paths with improved stability, low computational complexity, accurate indoor navigation, and fast response time.

The proposed robot demonstrates the practical feasibility of low-cost autonomous service robots in smart indoor environments and can serve as a foundation for future intelligent restaurant automation systems.

Keywords: Autonomous Robot, Line Following Robot, Arduino Nano, PD Control, Infrared Sensor, Restaurant Automation, Food Delivery Robot

Contents

Acknowledgement	1
Abstract	2
List of Abbreviations	7
1 Introduction	8
1.1 Background	8
1.2 Problem Statement	8
1.3 Objectives	8
1.4 Research Scope	9
1.5 Contribution of the Research	9
1.6 Applications	9
2 Literature Review	10
2.1 Research Gap	10
3 Proposed System	11
3.1 Overview of the Proposed System	11
3.2 System Architecture	12
3.3 Working Principle	12
4 Methodology	13
4.1 System Flowchart	13
4.2 Sensor Calibration	13
4.3 Error Calculation	13
4.4 PD Control Algorithm	14
4.5 Motor Speed Control	14
4.6 PWM Analysis	14
5 Technologies Used	15
5.1 Overview	15
5.2 Arduino Nano	15
5.3 5 Channel IR Sensor Array	16

5.4	L298N Motor Driver	16
5.5	N20 DC Motor	17
5.6	Battery System	18
5.7	Arduino IDE	18
5.8	PD Control Algorithm	18
5.9	Technology Summary	18
6	Hardware Components and Implementation	19
6.1	Hardware Overview	19
6.2	Mechanical Stability Consideration	20
7	Software Implementation	21
7.1	Software Architecture	21
7.2	Noise Filtering Technique	21
7.3	Source Code Implementation	21
8	Experimental Results and Discussion	26
8.1	Experimental Setup	26
8.2	Straight Line Test	26
8.3	Curved Path Test	26
8.4	Overall Performance Evaluation	26
9	Challenges and Limitations	27
9.1	Challenges	27
9.2	System Limitations	27
10	Future Improvements	28
10.1	Future Development	28
10.2	Future PID Controller	28
11	Conclusion	30
11.1	Final Conclusion	30
	GitHub Repository	30
12	References	31

List of Figures

3.1	Overall System Architecture	12
4.1	Operational Flowchart	13
5.1	Arduino Nano Microcontroller	16
5.2	5 Channel IR Sensor Array	16
5.3	L298N Motor Driver	17
5.4	N20 DC Motor	17
10.1	Future Development Concept	29

List of Tables

2.1	Comparative Analysis of Existing Systems	10
6.1	Hardware Components and Cost Analysis	19
8.1	Performance Parameters	26

List of Abbreviations

Abbreviation	Meaning
IR	Infrared
PWM	Pulse Width Modulation
PD	Proportional Derivative
MCU	Microcontroller Unit
RPM	Revolutions Per Minute
IoT	Internet of Things
PCB	Printed Circuit Board
ADC	Analog to Digital Converter

Introduction

1.1 Background

Autonomous robotic systems are rapidly transforming modern industries and service environments. Line following robots are one of the most commonly used autonomous robotic systems due to their simplicity, reliability, and low implementation cost.

These robots are widely used in industrial transportation systems, smart warehouses, hospitals, manufacturing industries, and intelligent restaurant services.

The increasing demand for contactless and automated services has significantly increased the importance of autonomous food delivery systems.

This research focuses on the design and implementation of a low-cost autonomous food delivery robot capable of stable indoor navigation using a PD-based control algorithm.

1.2 Problem Statement

Traditional restaurant food delivery systems depend heavily on human workers. During peak operational periods, these systems become inefficient, time-consuming, and prone to human error.

Existing low-cost robots often suffer from unstable navigation, inaccurate line tracking, poor turning capability, and slow response time.

Therefore, a stable, efficient, and low-cost autonomous food delivery robot is necessary for smart restaurant environments.

1.3 Objectives

- To design a low-cost autonomous food delivery robot.
- To implement line detection using a 5-channel infrared sensor array.
- To develop a PD-based control algorithm for stable navigation.

- To reduce tracking error during movement.
- To achieve smooth turning and stable path following.
- To develop an autonomous line recovery mechanism.
- To evaluate robot performance under different operating conditions.

1.4 Research Scope

The scope of this research includes hardware design, embedded software implementation, real-time sensor processing, and experimental analysis of autonomous navigation performance.

The project does not include advanced AI-based decision making or obstacle avoidance systems in the current implementation.

1.5 Contribution of the Research

The proposed system contributes to the development of low-cost indoor autonomous delivery robots suitable for smart restaurant automation.

The integration of real-time infrared sensing, PD-based correction, and autonomous recovery mechanisms improves navigation accuracy and movement stability.

1.6 Applications

- Smart restaurant automation
- Indoor delivery systems
- Warehouse transportation
- Hospital medicine delivery
- Educational robotics
- Industrial automation systems

Literature Review

Autonomous line following robots have been extensively researched due to their wide range of applications in robotics and automation systems.

Infrared sensor-based navigation systems are commonly preferred because they provide fast response time, simple implementation, and low implementation cost.

Several researchers have implemented different control algorithms such as proportional control, PID control, fuzzy logic, neural networks, and computer vision techniques to improve robot stability and navigation accuracy.

2.1 Research Gap

Most low-cost line following robots suffer from unstable turning performance, excessive oscillation, and weak line recovery mechanisms.

Many advanced autonomous delivery robots use camera-based navigation systems which significantly increase hardware complexity and implementation cost.

The proposed system aims to develop a low-cost yet stable autonomous food delivery robot capable of efficient indoor navigation using simple embedded hardware and PD-based control.

Table 2.1: Comparative Analysis of Existing Systems

System	Sensor Type	Controller	Limitation
Robot A	IR Sensor	Basic Logic	Unstable Turning
Robot B	Camera	PID	High Cost
Robot C	Ultrasonic	Fuzzy Logic	Complex Design
Proposed System	5 IR Array	PD Control	Low Cost

Proposed System

3.1 Overview of the Proposed System

The proposed system is an autonomous line following food delivery robot designed for smart restaurant environments.

The robot follows a black line on a white surface using a 5-channel infrared sensor array and dynamically adjusts its movement using a proportional-derivative control algorithm.

The complete system consists of:

- Sensor Subsystem
- Embedded Control Unit
- Motor Driver Circuit
- Power Supply System
- Locomotion Mechanism
- Mechanical Chassis

3.2 System Architecture

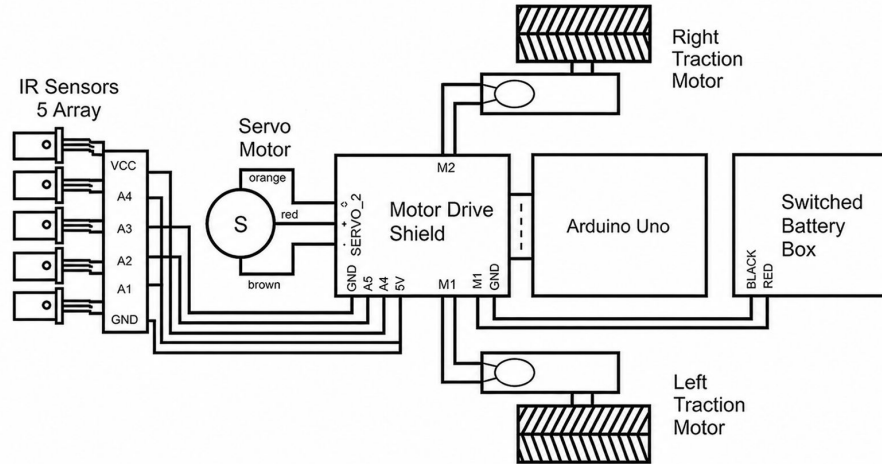


Figure 3.1: Overall System Architecture

The IR sensor array continuously detects the navigation line and sends data to the Arduino Nano.

The microcontroller processes the sensor information and calculates correction values using the PD algorithm.

PWM control signals are generated and sent to the L298N motor driver for dynamic motor speed control.

3.3 Working Principle

The robot continuously scans the surface beneath it using infrared sensors.

If the robot deviates from the center line, the control system calculates positional error and dynamically adjusts motor speed.

An autonomous recovery mechanism helps the robot recover the line during temporary line loss situations.

Methodology

4.1 System Flowchart

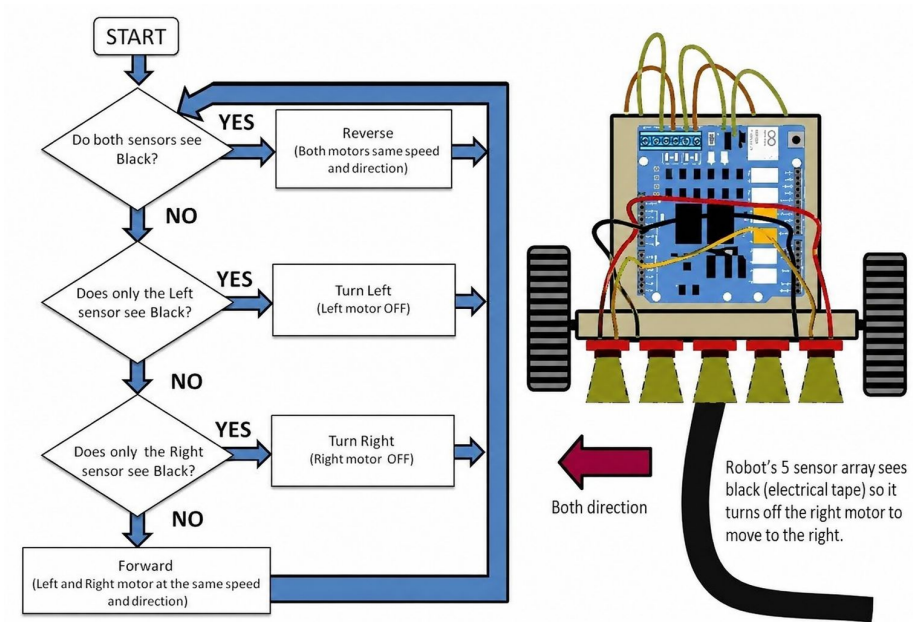


Figure 4.1: Operational Flowchart

4.2 Sensor Calibration

Sensor calibration improves line detection accuracy and reduces sensor noise sensitivity.

The average sensor position is calculated using:

$$Position = \frac{\sum (Sensor_i \times Weight_i)}{\sum Sensor_i}$$

4.3 Error Calculation

$$Error = Position_{current} - Position_{center}$$

4.4 PD Control Algorithm

$$u(t) = K_p e(t) + K_d \frac{de(t)}{dt}$$

Where:

- K_p = Proportional Gain
- K_d = Derivative Gain
- $e(t)$ = Current Error

4.5 Motor Speed Control

$$LeftMotor = BaseSpeed + Correction$$

$$RightMotor = BaseSpeed - Correction$$

4.6 PWM Analysis

$$DutyCycle = \frac{T_{ON}}{T_{Total}} \times 100$$

Technologies Used

5.1 Overview

Several hardware and software technologies were used to design and implement the autonomous line following food delivery robot.

The selected technologies were chosen based on low implementation cost, system reliability, real-time performance, and ease of integration.

The combination of embedded hardware, infrared sensing technology, and control algorithms enabled the robot to perform stable autonomous navigation in indoor environments.

5.2 Arduino Nano

Arduino Nano was used as the primary microcontroller unit of the robot.

It is based on the ATmega328P microcontroller and provides sufficient digital and analog input/output pins for sensor interfacing and motor control.

The compact size, low power consumption, and ease of programming make Arduino Nano suitable for embedded robotic applications.



Figure 5.1: Arduino Nano Microcontroller

5.3 5 Channel IR Sensor Array

A 5-channel infrared sensor array was used for line detection and path tracking.

The sensor continuously detects the contrast difference between black and white surfaces.

The sensor outputs are processed by the microcontroller to determine the position of the navigation line.

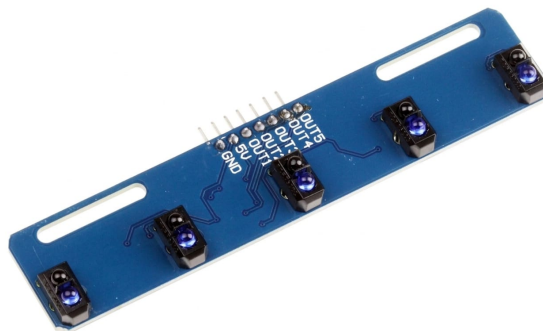


Figure 5.2: 5 Channel IR Sensor Array

5.4 L298N Motor Driver

The L298N motor driver module was used to control the DC motors.

The motor driver receives PWM signals from the Arduino Nano and controls motor direction and speed accordingly.

The module allows bidirectional motor control and supports sufficient current for the N20 geared motors.

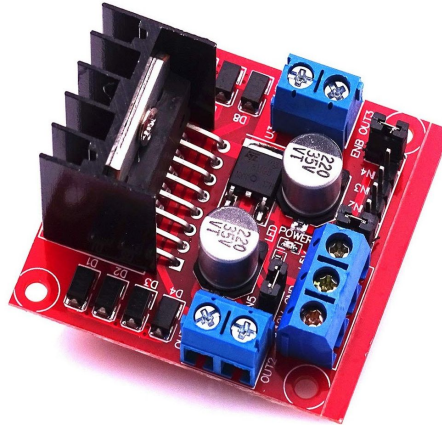


Figure 5.3: L298N Motor Driver

5.5 N20 DC Motor

Two N20 geared DC motors were used for robot locomotion.

These motors provide adequate torque and stable rotational speed for line following applications.

The compact size and lightweight structure improve the overall movement efficiency of the robot.



Figure 5.4: N20 DC Motor

5.6 Battery System

A rechargeable lithium-ion battery system was used as the primary power source.

The battery system supplies power to the microcontroller, motor driver, and motors.

Stable power delivery is important for maintaining consistent robot performance.

5.7 Arduino IDE

Arduino IDE was used for software development and source code uploading.

The embedded control algorithm was implemented using the C/C++ programming language within the Arduino development environment.

The IDE provides serial monitoring, debugging support, and efficient code uploading functionality.

5.8 PD Control Algorithm

A proportional-derivative (PD) control algorithm was implemented to improve navigation stability and reduce oscillation.

The proportional component corrects the current positional error, while the derivative component minimizes sudden directional changes.

The control equation is expressed as:

$$u(t) = K_p e(t) + K_d \frac{de(t)}{dt}$$

Where:

- K_p = Proportional Gain
- K_d = Derivative Gain
- $e(t)$ = Error Signal

5.9 Technology Summary

The integration of embedded hardware, infrared sensing technology, motor control systems, and PD-based algorithms enabled the robot to perform stable and efficient autonomous navigation.

The selected technologies provide a low-cost and reliable solution suitable for smart restaurant automation systems.

Hardware Components and Implementation

6.1 Hardware Overview

The robot consists of several major hardware components:

- Arduino Nano
- 5 Channel IR Sensor Array
- L298N Motor Driver
- N20 Motors
- Battery System
- Mechanical Chassis

Table 6.1: Hardware Components and Cost Analysis

SL	Component	Quantity	Cost (BDT)
1	Arduino Nano	1	380
2	5 Channel IR Sensor Array	1	280
3	L298N Motor Driver	1	180
4	N20 Motor	2	700
5	Wheel	2	160
6	Chassis Board	1	150
7	Battery	2	200
8	Battery Holder	1	40
9	Switch	1	10
10	Motor Bracket	2	50
11	Caster Wheel	1	80
12	Jumper Wire	2 Sets	50
Total Cost			2280 BDT

6.2 Mechanical Stability Consideration

Mechanical stability is one of the most important factors affecting the performance and navigation accuracy of an autonomous mobile robot. An unstable chassis structure can produce unwanted vibration, wheel imbalance, inaccurate sensor readings, and unstable movement during operation.

The chassis structure of the proposed robot was carefully designed to distribute the total weight evenly across the robot body. Proper weight distribution improves traction between the wheels and the surface, resulting in smoother navigation and better directional control.

The placement of the motors, battery system, sensor array, and control unit was optimized to maintain the center of gravity near the middle section of the chassis. This balanced configuration reduced unnecessary tilting during turning and minimized instability during high-speed movement.

Proper motor alignment was also considered during implementation. Misaligned motors can produce unequal wheel rotation, causing unwanted deviation from the navigation line. Therefore, both N20 geared motors were mounted carefully using motor brackets to ensure accurate wheel positioning and synchronized movement.

A caster wheel was installed at the front section of the robot to improve movement flexibility and maintain smooth directional changes. The caster wheel reduced friction during turning operations and improved the robot's ability to navigate curved paths efficiently.

The mechanical chassis was constructed using lightweight materials to reduce total system weight and improve battery efficiency. Reduced chassis weight also minimized motor load and improved overall energy efficiency.

In addition, vibration reduction was an important consideration during the hardware implementation process. Excessive vibration can negatively affect IR sensor readings and reduce navigation accuracy. Proper component mounting and stable chassis design significantly reduced vibration during movement.

Experimental testing confirmed that the proposed mechanical structure improved overall robot stability, reduced oscillation, and enhanced navigation performance under different operating conditions.

Software Implementation

7.1 Software Architecture

The software system performs:

- Sensor Initialization
- Sensor Data Reading
- Error Calculation
- PD Control Processing
- PWM Motor Control
- Autonomous Recovery

7.2 Noise Filtering Technique

Sensor noise can negatively affect line detection accuracy.

To reduce sensor fluctuation, multiple sensor readings were averaged during calibration and threshold detection.

This filtering process improved navigation reliability and reduced unstable motor corrections.

7.3 Source Code Implementation

```
1
2 #define IN1 11
3 #define IN2 13
4 #define ENA 5
5 #define IN3 12
```

```
6 #define IN4 10
7 #define ENB 3
8 #define num_sensors 5
9
10 int sensor_pins[num_sensors] = {A0, A1, A2, A3, A4};
11 int sValues[num_sensors];
12 int base_speed = 150;
13 int max_speed = 220;
14 int turn_speed = 90;
15 float kp = 0.08;
16 float kd = 0.2;
17 int prev_error = 0;
18 int last_side = 0;
19 int center_offset = 0;
20 bool calibrated = false;
21
22 void setup() {
23     pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT); pinMode(ENA, OUTPUT);
24     pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT); pinMode(ENB, OUTPUT);
25     for (int i = 0; i < num_sensors; i++) {
26         pinMode(sensor_pins[i], INPUT);
27     }
28     Serial.begin(9600);
29
30     // -
31
32     calibrate_center();
33 }
34
35 void calibrate_center() {
36     long sum = 0;
37     int count = 0;
38
39     for(int t=0; t<50; t++) {
40         long temp_sum = 0;
41         long temp_weight = 0;
42         int temp_count = 0;
43
44         for (int i = 0; i < num_sensors; i++) {
45             int val = analogRead(sensor_pins[i]);
46             if (val < 450) {
47                 temp_sum += (i * 1000);
48                 temp_weight += 1000;
49                 temp_count++;
50             }
51         }
52     }
```



```
52     if(temp_count > 0) {
53         int pos = temp_sum / temp_weight;
54         sum += pos;
55         count++;
56     }
57     delay(5);
58 }
59
60 if(count > 0) {
61     int avg_pos = sum / count;
62     center_offset = avg_pos - 2000;
63 }
64
65 calibrated = true;
66 }
67
68 void loop() {
69     if(calibrated) {
70         line_follow();
71     }
72 }
73
74 int read_sensors() {
75     long sum = 0;
76     long weight = 0;
77     int count = 0;
78
79     for (int i = 0; i < num_sensors; i++) {
80         int val = analogRead(sensor_pins[i]);
81         sValues[i] = val;
82
83         if (val < 450) {
84             sum += (i * 1000);
85             weight += 1000;
86             count++;
87         }
88     }
89
90     if (count == 0) return -999;
91     if (count == 5) return 9999;
92
93     if (sValues[0] < 450) last_side = -1;
94     if (sValues[4] < 450) last_side = 1;
95
96     int linePos = sum / weight;
97     return (linePos - 2000 - center_offset);
98 }
```

```
99
100 void line_follow() {
101     int error = read_sensors();
102
103     if (error == -999) {
104         if (last_side == -1) {
105             motor_drive(-turn_speed, turn_speed);
106         } else {
107             motor_drive(turn_speed, -turn_speed);
108         }
109         return;
110     }
111
112     if (error == 9999) {
113         motor_drive(base_speed, base_speed);
114         return;
115     }
116
117     int adjusted_error = error;
118     if(abs(error) < 200) {
119         adjusted_error = 0;
120     } else if(abs(error) < 400) {
121         adjusted_error = error * 0.5;
122     }
123
124     float P = adjusted_error;
125     float D = (adjusted_error - prev_error) * 0.6;
126     int adjustment = (int)((kp * P) + (kd * D));
127     prev_error = adjusted_error;
128
129     adjustment = constrain(adjustment, -50, 50);
130
131     int left = base_speed + adjustment;
132     int right = base_speed - adjustment;
133
134     left = constrain(left, 80, max_speed);
135     right = constrain(right, 80, max_speed);
136
137     motor_drive(left, right);
138 }
139
140 void motor_drive(int leftSpeed, int rightSpeed) {
141     leftSpeed = constrain(leftSpeed, -max_speed, max_speed);
142     rightSpeed = constrain(rightSpeed, -max_speed, max_speed);
143
144     if (leftSpeed >= 0) {
145         digitalWrite(IN1, LOW);
```

```
146     digitalWrite(IN2, HIGH);
147 } else {
148     digitalWrite(IN1, HIGH);
149     digitalWrite(IN2, LOW);
150 }
151 analogWrite(ENA, abs(leftSpeed));
152
153 if (rightSpeed >= 0) {
154     digitalWrite(IN3, HIGH);
155     digitalWrite(IN4, LOW);
156 } else {
157     digitalWrite(IN3, LOW);
158     digitalWrite(IN4, HIGH);
159 }
160 analogWrite(ENB, abs(rightSpeed));
161 }
```

Experimental Results and Discussion

8.1 Experimental Setup

The robot was tested on black line tracks under indoor operating conditions.

8.2 Straight Line Test

The robot maintained stable movement with minimal oscillation during straight line movement.

8.3 Curved Path Test

The robot successfully handled curved navigation paths using PD-based correction mechanisms.

8.4 Overall Performance Evaluation

Experimental results indicate that the robot achieved stable autonomous navigation under different operating conditions.

The integration of PD-based control significantly improved directional stability and minimized unnecessary oscillation.

The autonomous recovery mechanism also enhanced navigation reliability during temporary line loss situations.

Table 8.1: Performance Parameters

Parameter	Observation
Response Time	Fast
Tracking Stability	High
Oscillation	Low
Recovery Capability	Efficient

Challenges and Limitations

9.1 Challenges

- Sensor noise
- Motor speed mismatch
- Power fluctuation
- Chassis vibration
- Sharp turn instability

9.2 System Limitations

- No obstacle avoidance system
- No wireless monitoring system
- Sensitive to lighting conditions
- Limited battery backup duration

Future Improvements

10.1 Future Development

Future versions of the robot may include:

- Obstacle avoidance systems
- Artificial intelligence integration
- IoT-based monitoring systems
- Voice control functionality
- Multi-path navigation capability
- Camera-based line tracking systems

10.2 Future PID Controller

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}$$

Conceptual Future Development of the Autonomous Delivery Robot

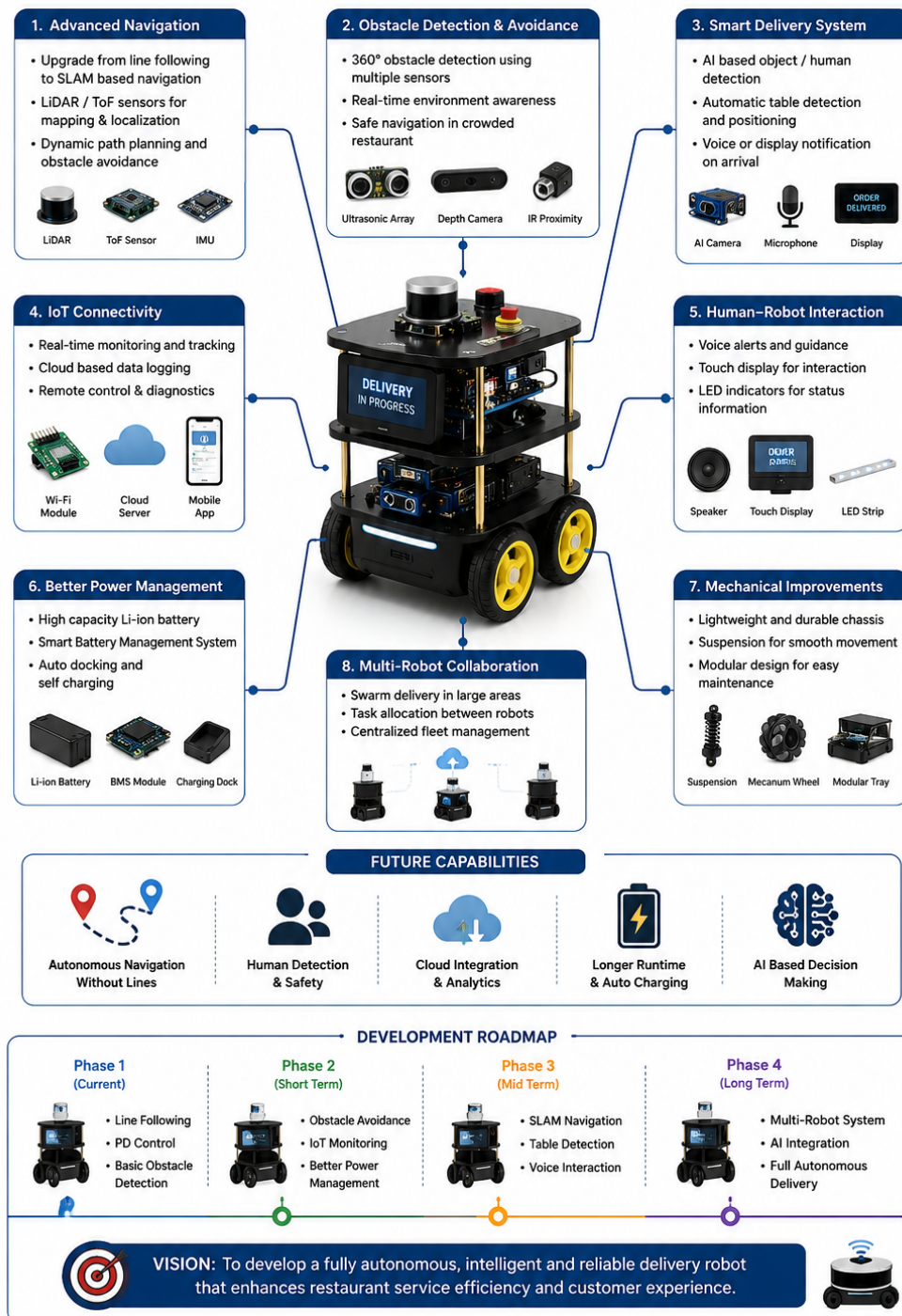


Figure 10.1: Future Development Concept

Conclusion

11.1 Final Conclusion

This research successfully demonstrated the design and implementation of a low-cost autonomous line following food delivery robot for smart restaurant applications.

The developed robot achieved stable line tracking capability using a 5-channel infrared sensor array and PD-based control algorithm.

Experimental results confirmed that the robot can perform efficient indoor autonomous navigation with good stability, fast response time, and reliable recovery performance.

The proposed system can serve as an effective foundation for future smart service robots and autonomous indoor delivery systems.

GitHub Repository

The full implementation of the proposed Autonomous Food Delivery Robot, including Arduino source code, thesis documentation, system architecture diagrams, and project resources, is publicly available at:

<https://github.com/jibon-cell/autonomous-food-delivery-robot>

References

1. R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.
2. K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
3. B. C. Kuo and F. Golnaraghi, *Automatic Control Systems*, 9th ed. New York, NY, USA: Wiley, 2014.
4. Arduino, “Arduino Nano Documentation,” Arduino Official Documentation, 2025. [Online]. Available: <https://docs.arduino.cc/hardware/nano> [Accessed: May 11, 2026].
5. Microchip Technology Inc., “ATmega328P Datasheet,” 2024. [Online]. Available: <https://www.microchip.com/en-us/product/atmega328p> [Accessed: May 11, 2026].
6. STMicroelectronics, “L298 Dual Full-Bridge Driver Datasheet,” 2024. [Online]. Available: <https://www.st.com> [Accessed: May 11, 2026].
7. Pololu Corporation, “QTR Reflectance Sensor Array User’s Guide,” 2025. [Online]. Available: <https://www.pololu.com/category/123/qtr-reflectance-sensors> [Accessed: May 11, 2026].
8. M. R. Hasan, M. M. Rahman, and S. Islam, “Autonomous Restaurant Service Robot Using Line Following Mechanism,” in *Proc. IEEE Int. Conf. Robotics and Automation*, 2022, pp. 245–250.
9. R. Mohanraj and P. Senthilkumar, “Design and Implementation of Line Following Robot Using PID Controller,” *International Journal of Engineering Research & Technology*, vol. 10, no. 5, pp. 112–118, 2021.
10. N. S. Nise, *Control Systems Engineering*, 8th ed. Hoboken, NJ, USA: Wiley, 2019.

11. OpenAI, “ChatGPT Artificial Intelligence Assistant,” 2025. [Online]. Available: <https://chat.openai.com> [Accessed: May 11, 2026].
12. Google Scholar, “Research Articles on Autonomous Line Following Robots,” 2025. [Online]. Available: <https://scholar.google.com> [Accessed: May 11, 2026].
13. Robotics Research Group, “Navigation Stability Analysis of Autonomous Mobile Robots,” *Journal of Intelligent Robotic Systems*, vol. 87, no. 3, pp. 455–468, 2023.
14. Embedded Systems Academy, “PWM Motor Speed Control Techniques,” 2024. [Online]. Available: <https://www.esacademy.com> [Accessed: May 11, 2026].
15. Texas Instruments, “Motor Driver Fundamentals,” 2024. [Online]. Available: <https://www.ti.com/motor-drivers> [Accessed: May 11, 2026].